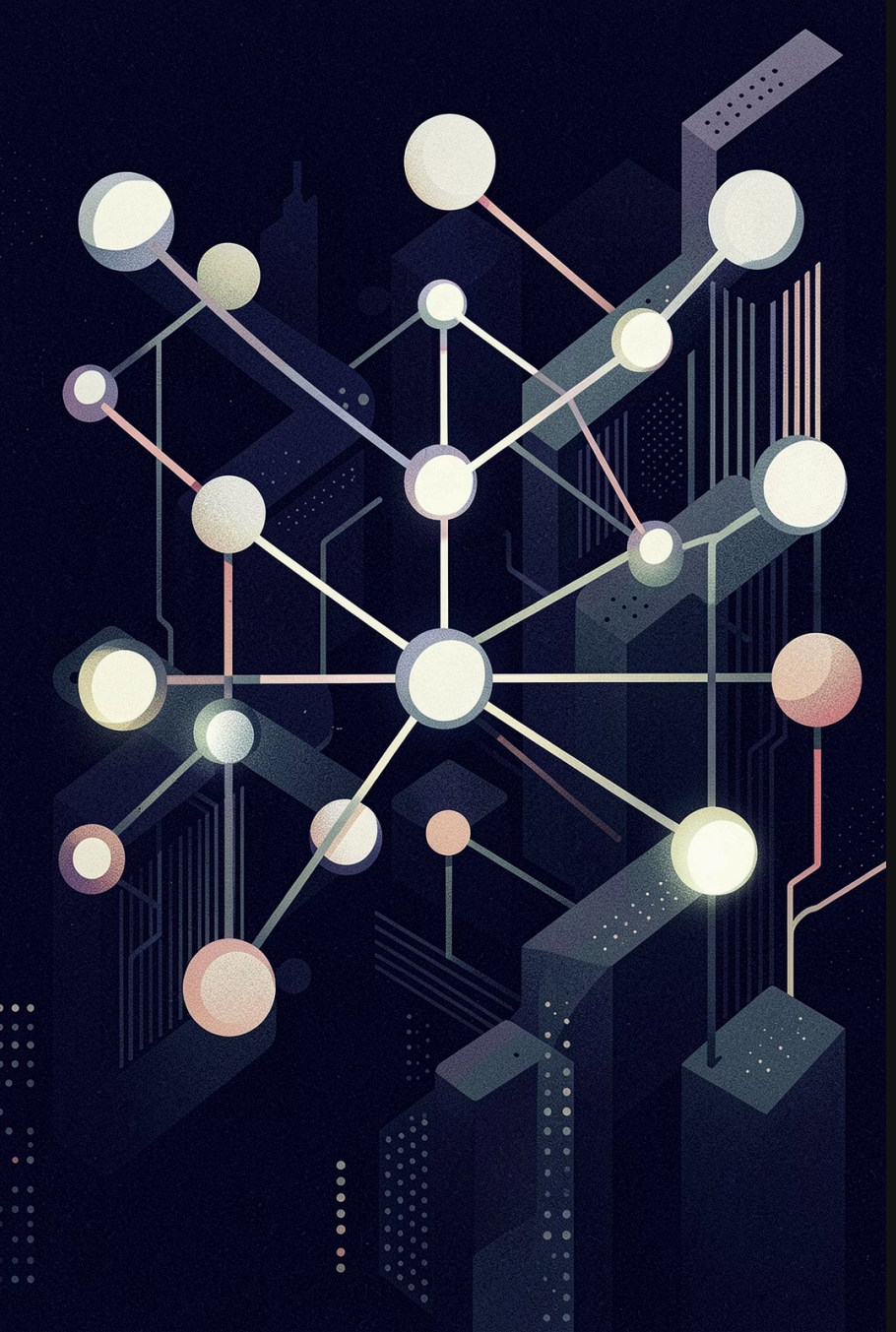


Building Useful AI Agents with Agno + OpenRouter

A practical workshop on modern agent engineering — from core concepts to production-ready systems.

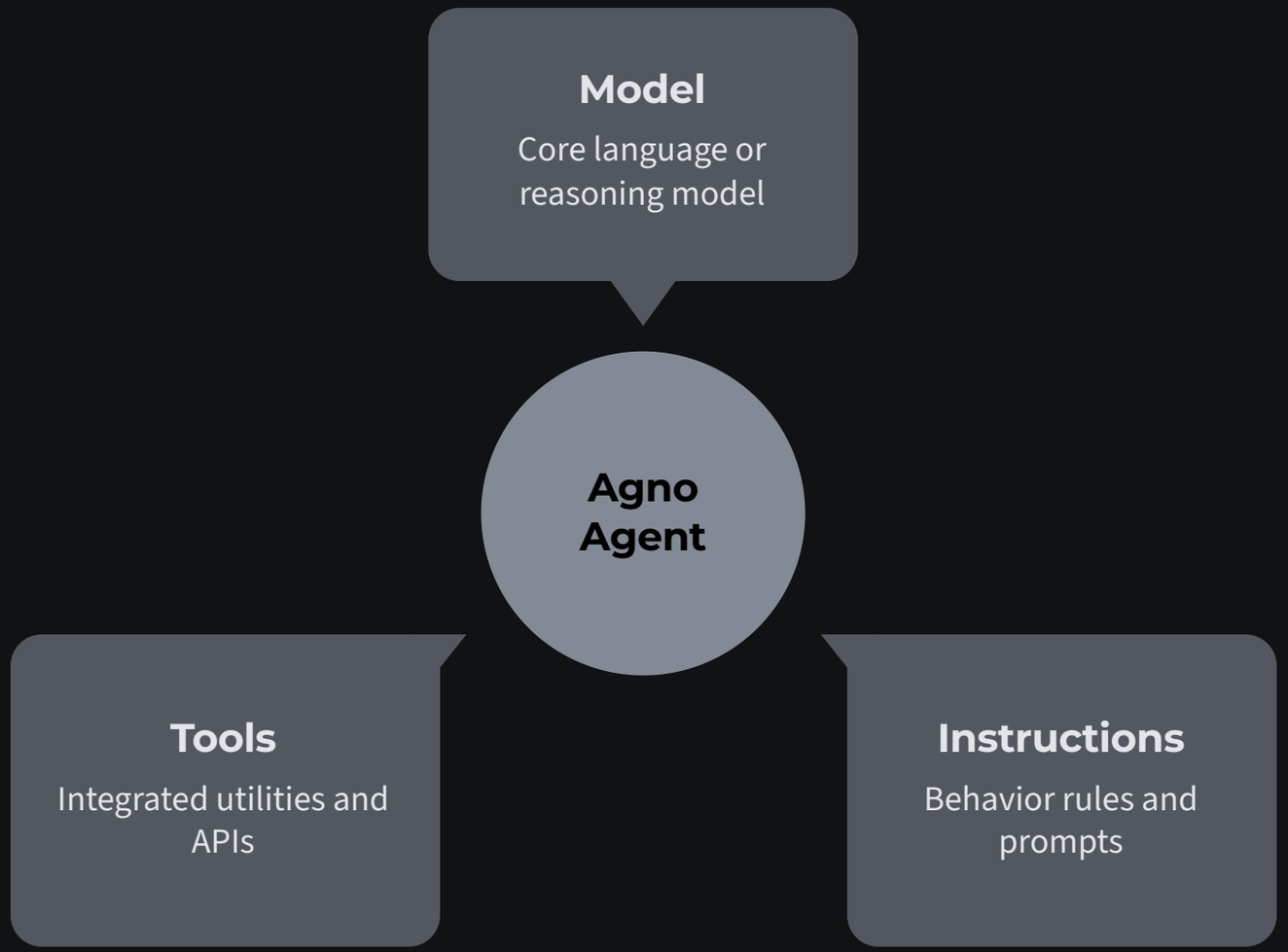
DEVELOPER WORKSHOP

AGENT ENGINEERING

01 — CORE CONCEPT

What is Agno?

Agno is a **lightweight, model-agnostic AI agent framework** with a fast runtime and batteries-included philosophy — Agents, Tools, Memory, Knowledge, and Workflows built in.



What is Agno? Cont'd

Model, instructions, and tools combine into a single Agent — the core unit of Agno.

Model Agnostic

Swap models without changing agent logic

Fast Runtime

Minimal overhead, maximum throughput

Batteries Included

Tools, memory, knowledge, storage built-in

Building Your First Agent

Building Your First Agent... Cont'd

Every agent follows this core loop — input in, structured response out.

```
from agno.agent import Agent
from agno.models.openai import OpenAIChat

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    instructions="You are a helpful assistant.",
    markdown=True
)

agent.print_response("What is AI?")
```

Three Building Blocks

Model

Any LLM via OpenRouter or a direct provider

Instructions

System prompt defining role and behavior

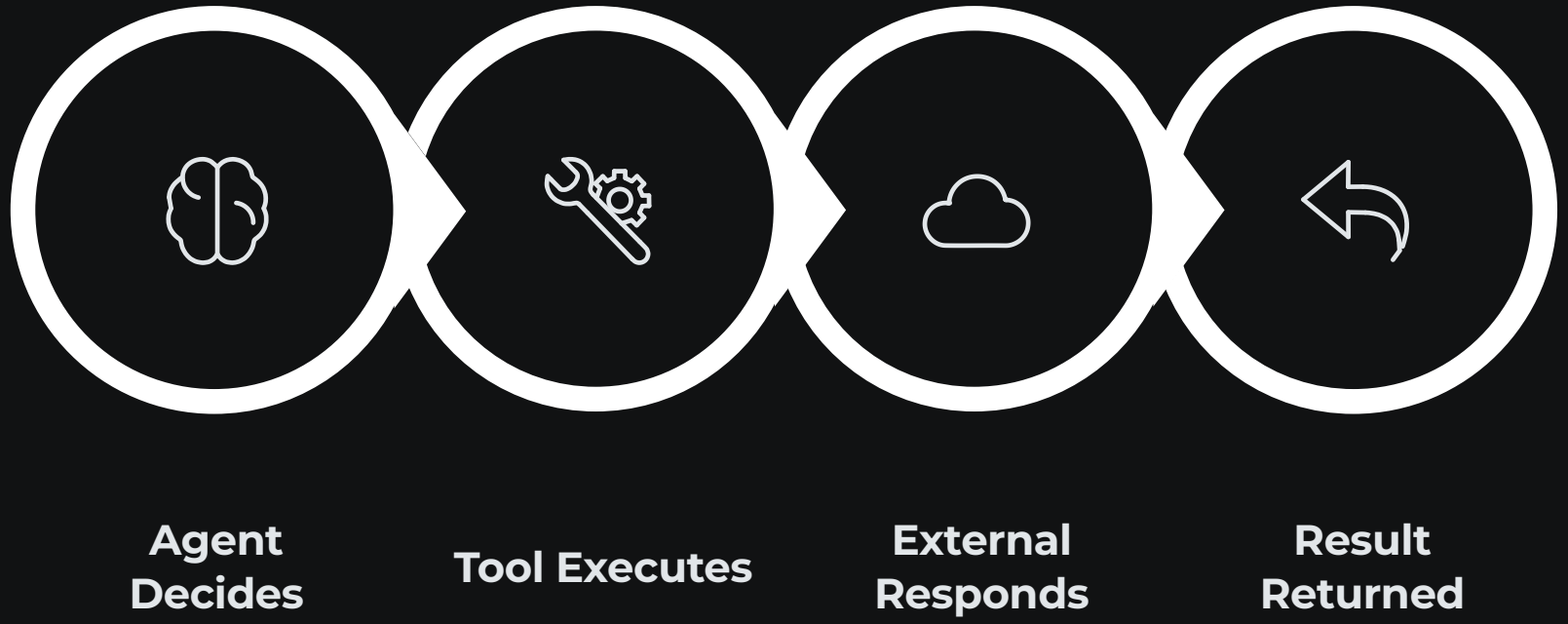
Response

Streaming or batched output, auto-formatted

03 — CAPABILITIES

Tools: Turning Assistants Into Agents

Tools let agents **act in the real world** — calling APIs, querying databases, and executing code. This is what separates agents from chatbots.



Tools... Cont'd

Tools bridge the agent and external systems, enabling real-world action.

```python
from agno.tools import tool


def get_weather(city: str) -> str:
    """Fetch current weather for a city."""
    return f"Weather in {city}: 72°F, Sunny"

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    tools=[get_weather],
    show_tool_calls=True
)
```

Tool Types

Built-in

Search, code, file I/O

Custom

Any Python function via @tool

External APIs

REST, GraphQL, databases

04 — RELIABILITY

Structured Outputs

Raw text is unpredictable. **Structured outputs** give you reliable, machine-readable responses using **Pydantic** schemas — essential for downstream pipelines.

❌ Raw Text Output

```
The city is Paris.
Population is about 2.1 million.
Capital of France.
```

✅ Structured JSON

```
{
  "city": "Paris",
  "population": 2100000,
  "country": "France",
  "is_capital": true
}
```

Pydantic Integration

```
from pydantic import BaseModel

class CityInfo(BaseModel):
    city: str
    population: int
    country: str
    is_capital: bool

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    response_model=CityInfo
)
```

ⓘ The agent auto-validates output against the schema — no parsing, no regex.

05 — COLLABORATION

Teams: Specialized Agents Working Together

Complex tasks need **specialized roles**. Teams let agents delegate subtasks, collaborate, and produce higher-quality results than any single agent.

Teams... Cont'd

Each agent owns a role — the team orchestrates handoffs automatically.

```
from agno.team import Team

researcher = Agent(
    name="Researcher",
    instructions="Find and verify facts."
)

writer = Agent(
    name="Writer",
    instructions="Write clear summaries."
)

team = Team(
    agents=[researcher, writer],
    model=OpenAIChat(id="gpt-4o")
)
```

Why Teams Win

- **Specialization** — each agent has a focused role
- **Delegation** — complex tasks split automatically
- **Quality** — focused agents outperform generalists

06 — CONTEXT

Memory: Context That Persists

Memory lets agents **remember conversations, preferences, and history** — enabling personalization and multi-turn coherence across sessions.

Memory Types

1

Session Memory

Short-term context within a single conversation

2

Persistent Memory

Long-term storage across sessions and users

3

User Memory

Personalized preferences and behavioral patterns

```
agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    memory=True,        # Enable session memory
    add_history_to_messages=True,
    num_history_responses=5
)

# Agent remembers prior turns
agent.print_response("My name is Alex")
agent.print_response("What is my name?")
# → "Your name is Alex"
```

Memory is the difference between a stateless API call and a coherent assistant.

07 — INFORMATION

Knowledge: External Information Sources

Knowledge bases give agents access to **documents, databases, and external data** — distinct from memory, which stores conversation history.

Knowledge... Cont'd

Knowledge is queried at runtime — the agent retrieves only what's relevant to the current task.

Knowledge vs. Memory

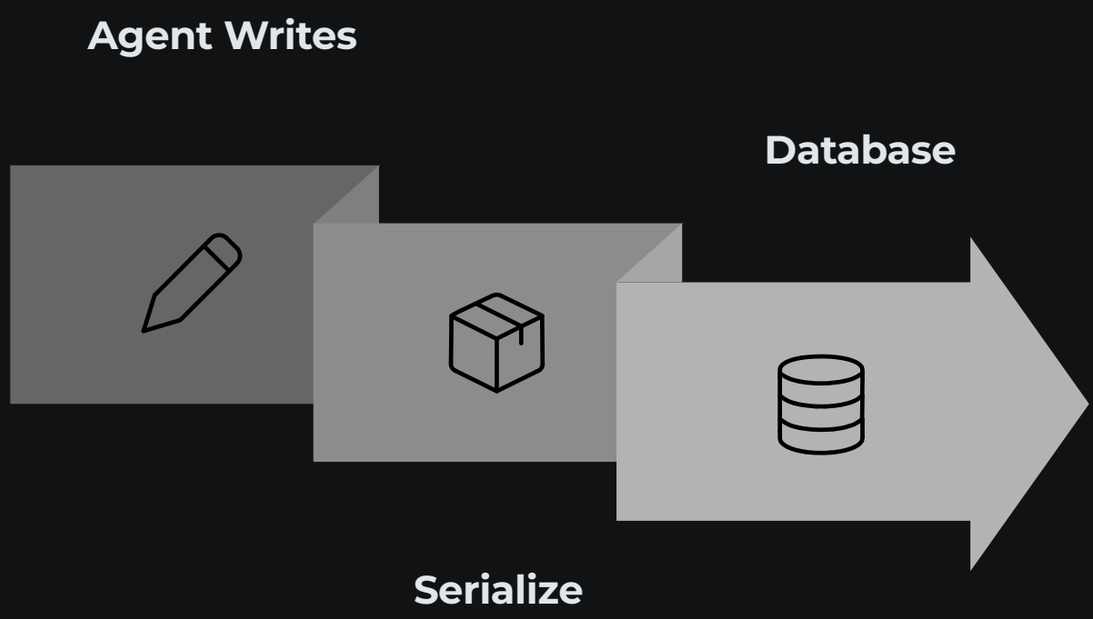
Memory	Knowledge
Conversation history	Documents & data
User-specific	Shared across users
Short + long term	Static + searchable
Built from chat	Loaded from files/DB

```
from agno.knowledge import KnowledgeBase

kb = KnowledgeBase(
    vector_db=...,
    documents=["docs/", "manuals/"]
)

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    knowledge=kb,
    search_knowledge=True
)
```

08 — PERSISTENCE

Storage: Production-Grade State Management

Production agents run across sessions, users, and restarts. **Storage** persists agent state, memory, and knowledge to a database — enabling long-running, reliable systems.



Storage... Cont'd

Storage ensures no state is lost — agents resume exactly where they left off.

Why Storage Matters

Session Recovery

Resume after crashes or restarts

Multi-User Scaling

Isolate state per user securely

Audit & Debug

Full trace of agent decisions

```
from agno.storage.sqlite import SqliteStorage

storage = SqliteStorage(
    table_name="agent_sessions",
    db_file="sessions.db"
)

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    storage=storage,
    read_chat_history=True
)
```

09 — ORCHESTRATION

Workflows: Multi-Step Agent Orchestration

Workflows connect agents, tools, memory, and knowledge into **structured task pipelines** — orchestrating complex, multi-step execution with full traceability.

Workflows... Cont'd

Workflows orchestrate every component — from input to final output — in a structured, traceable pipeline.

```
from agno.workflow import Workflow

workflow = Workflow(
    name="Research Pipeline",
    steps=[
        research_step,
        analyze_step,
        validate_step,
        publish_step
    ]
)

workflow.run(query="AI safety trends")
```

Workflow Benefits

- **Orchestration** — chain agents and tools in sequence
- **Error Handling** — retry logic and fallback steps
- **Observability** — trace every step and decision